

Assignment 3

Paul Kabulu}

KBLPAU002

1. Introduction

Encoder–decoder LSTM models are commonly used for sequence-to-sequence tasks like machine translation and text-to-speech. A key limitation is the information bottleneck, where the whole input sequence has to be compressed into a single fixed-size vector.

This report looks at three different decoder setups for the Grapheme-to-Phoneme (G2P) task. The main aim is to see whether attention improves performance compared to simpler fixed approaches.

2. Architecture Design

All models share the same encoder and embedding layers. The only difference is how the decoder uses information from the encoder.

Setup 1: Bottleneck Baseline (No Context)

The encoder produces a final hidden state $(h_{\text{enc}}, c_{\text{enc}})$ which is used to initialise the decoder. After that, the decoder does not get any more information from the encoder.

So basically, all input information has to be squeezed into one vector, which is not ideal for longer or more complex words.

Setup 2: Fixed Context Vector

In this version, the final encoder hidden state $z = h_{\text{enc}}$ is passed into every decoding step. The LSTM gates become:

$$\begin{aligned} f_t &= \sigma(W_f y_t + U_f h_{t-1} + V_f z) \\ i_t &= \sigma(W_i y_t + U_i h_{t-1} + V_i z) \\ o_t &= \sigma(W_o y_t + U_o h_{t-1} + V_o z) \\ \tilde{c} * t &= \tanh(W_c y_t + U_c h * t - 1 + V_c z) \end{aligned}$$

This means the decoder always has access to the same global summary of the input sequence. It helps a bit, but the information is still doesn't change over time.

Setup 3: Dynamic Attention

Here, the model computes a different context vector at each time step using dot-product attention:

$$e_{t,j} = h_t^T h_{\text{enc},j}$$

$$\alpha_t = \text{softmax}(e_t)$$

$$z_t = \sum_j \alpha_{t,j} h_{\text{enc},j}$$

This replaces the fixed context with a dynamic one, so the decoder can focus on different parts of the input depending on what it is currently predicting.

Implementation Note: EOS Truncation

During batch decoding, some sequences finish earlier than others when a token is produced. For evaluation, I truncate predictions at the first token so that padding of leftover tokens don't affect the results. This keeps the phomeme error rate (PER) calculation consistent.

3. Hyperparameter Tuning

A grid search was done on Setup 1 using 27 combinations (3 learning rates × 3 embedding sizes × 3 hidden sizes). Each model was trained for 5 epochs due to time constraints.

Learning Rate	Embedding Size	Hidden Size	Validation PER
1e-5	128	256	0.6483
1e-4	128	256	0.2576
1e-3	64	256	0.1373

Chosen configuration: lr = 1e-3, embedding = 64, hidden = 256

Reasoning:

- 1e-3 worked best and trained noticeably faster.
- Embedding size 64 gave the best balance, 32 underfit, while 128 did not improve performance.
- A hidden size of 256 worked best majority of the time, showing that a larger model capacity helps it keep sequence information despite the bottleneck.

4. Experimental Results

All models were trained under identical conditions (same seed, batch size, and early stopping with patience = 5).

Training and Validation Behaviour

Training loss looked basically the same for all models, going from about 1.0 down to ~0.1. So capacity-wise, models are decent. The main differences appear in validation performance:

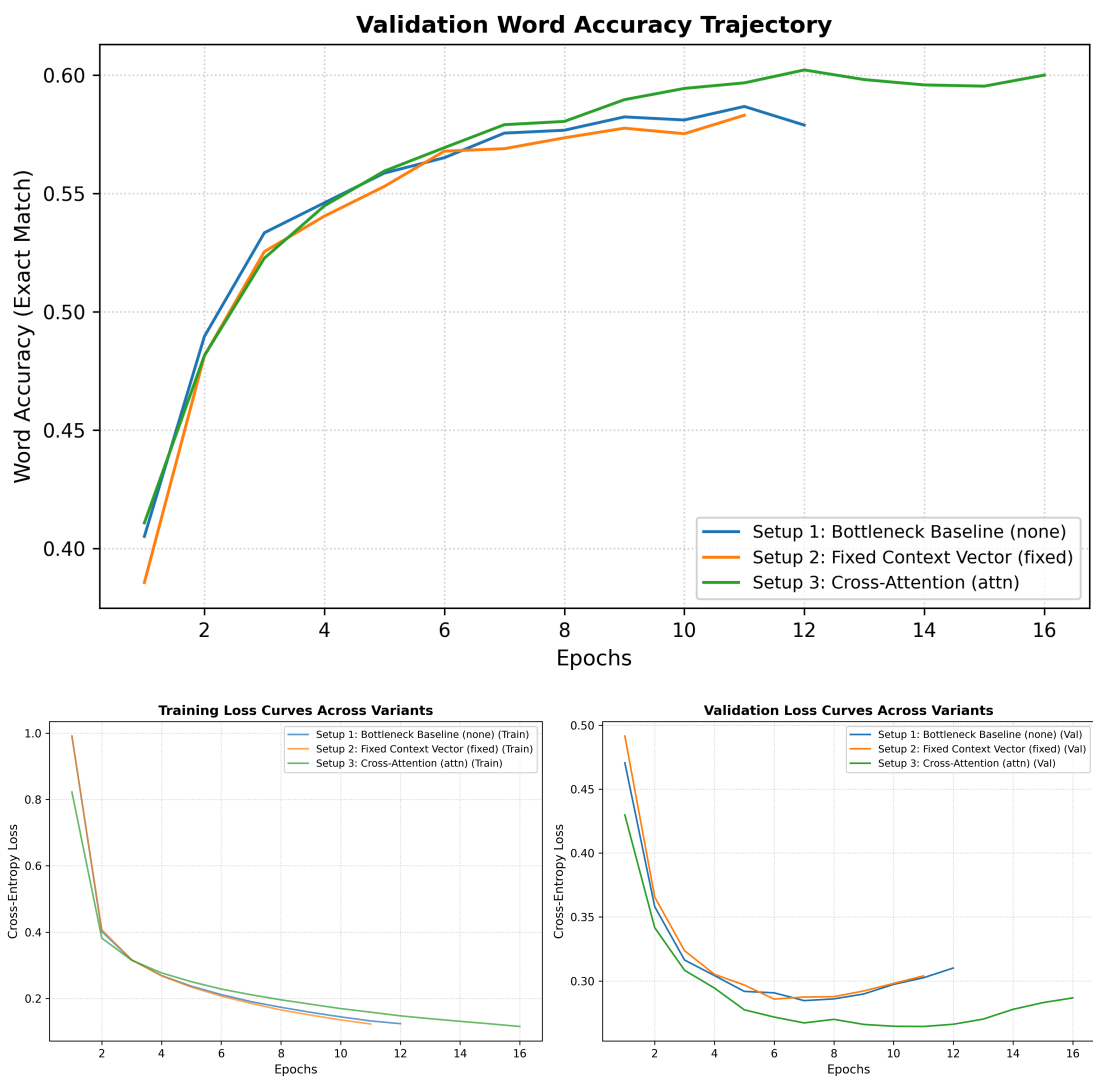
- **Setup 1:** Improves early but starts overfitting after around ~epoch 7.
- **Setup 2:** slightly better but still similar pattern.
- **Setup 3:** keeps improving longer and reaches the best validation loss.

Validation accuracy also follows the same trend, with Setup 3 getting around 60%, while the others remian around 57–58%.

Test Set Performance

Setup	PER	Word Accuracy
Setup 1 (no context)	0.1352	57.21%
Setup 2 (fixed context)	0.1326	58.10%
Setup 3 (attention)	0.1173	59.91%

Setup 2 only gives a small improvement over the baseline. Setup 3 gives the clearest gain, with the lowest error rate and best accuracy overall.



5. Why Attention Helps

The main problem with Setup 1 is that all input information is compressed into one vector, which obviously loses detail, especially for longer/complex words.

Setup 2 is a bit better because it keeps reusing the same context vector, but it's still static, so it can't adapt depending on what the decoder is currently generating.

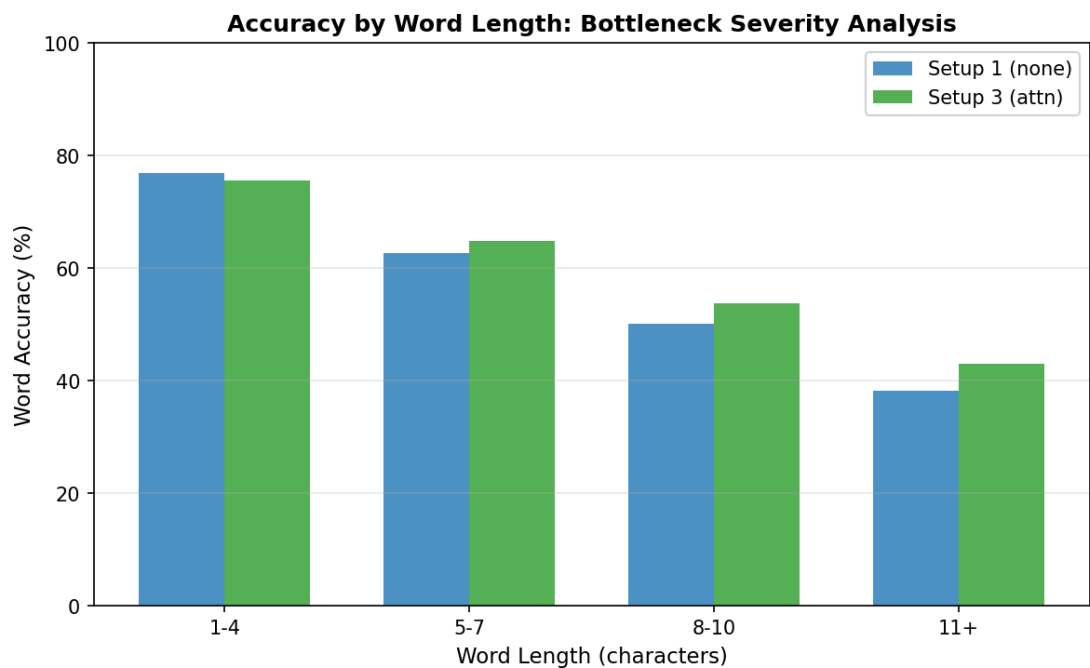
Setup 3 fixes this by recomputing the context at every step, so the model can focus on different parts of the input when needed. This basically reduces the severity of the bottleneck.

6. Character Lengths

To check how performance changes with word length, results were grouped by character length.

Length	Setup 1	Setup 2	Setup 3	Improvement (1->3)
1-4 chars	76.9%	75.4%	75.5%	-1.4%
5-7 chars	62.7%	64.4%	64.9%	+2.2%
8-10 chars	50.0%	50.6%	53.8%	+3.7%
11+ chars	38.1%	37.4%	43.0%	+4.9%

The results show a clear trend: Setup 1 (baseline bottleneck) performs much worse on longer words, falling from 76.9% accuracy on short words to 38.1% on words with 11+ characters, a drop of 38.8%. Setup 3 (with attention) also declines, but less sharply, from 75.5% to 43.0%, a 32.5% drop. The smaller decline (6.3 less) suggests that attention helps reduce the bottleneck problem, especially for longer sequences. Attention gives little improvement on short words (1.4%) but much larger gains on long words (4.9%), where the task is harder.



8. Conclusion

Overall, the results show that attention mechanisms improve performance for G2P tasks using encoder–decoder LSTMs.

- Baseline struggles especially on longer inputs
- Fixed context helps slightly but is still limited
- The attention model consistently performs best, especially on harder cases

The improvements aren't that noticeable, but they are consistent and match what you would expect from reducing the information bottleneck problem in sequence models.